IoC and Reflection

November 2026

Module 4 overview

- Module 4: Advanced Java
- Look under the hood of Java and frameworks
- Goal: understand why things work
- One growing project: your own mini-CDI container

Topics per day

Day	Morning topics	Afternoon project
1	IoC, DI, annotations, reflection, type erasure	Registry and constructor injection
2	Source to classpath, class loading, Maven	Classpath scanning, interfaces, plugin beans
3	Functional Java: lambdas, streams, `Optional`, records	Refactor the container, lazy `Provider<T>`
4	CDI in depth: scopes, lifecycle, proxies, interceptors	Lifecycle, scopes, `@Logged`
5	Concurrency and async	Thread-safe container
6	Memory, garbage collection, profiling	Profile and improve the container
7	Finish the project	Presentations and quiz

Time planning

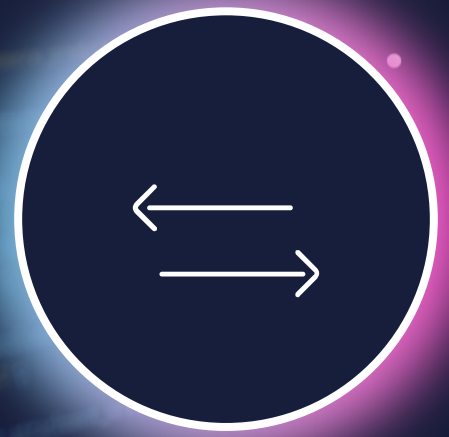
Starting time: 9.30

Breaks: ???

End time: 16.30

Learning goals

- Explain IoC and dependency injection
- Create and read your own annotations
- Inspect and call code with reflection
- Explain type erasure and its effects
- Build the core of a mini-CDI container



Section 1

Inversion of control and dependency injection

Welcome to advanced Java

Last block: database and network APIs

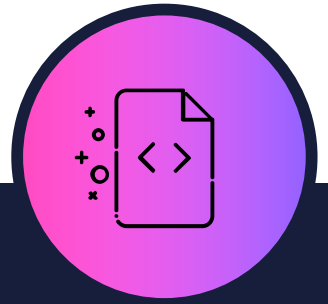
This block: Java and frameworks under the hood

Focus on why, not only that

The common thread: a mini-CDI container

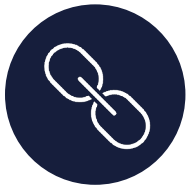
- Every afternoon we extend our own container
- Day 1: a registry that creates and connects objects
- Later days: more features on the same code base

The problem



```
public class OrderService {
    private final PaymentService payment = new PaymentService();
    private final EmailSender email = new EmailSender();

    public void placeOrder(Order order) {
        payment.charge(order);
        email.send(order.getCustomerEmail(), "Thanks!");
    }
}
```

Consequences of using `new`



Tight coupling to concrete classes

Hard to test: no fake `EmailSender`

Hard to change: edit every caller

Every user must know how to build a `PaymentService`

Inversion of Control

Your code no longer decides who creates what

Something else does that for you

”Don’t call us, we call you”

IoC is broader than DI

Servlets: the container calls your code

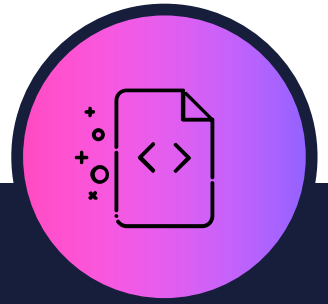
JUnit: the framework calls your test

JAX-RS: you never call your own resource

Dependency injection

- An object receives its dependencies
- It does not create them itself
- One way to apply IoC

Three forms of injection



```
// Constructor injection
public OrderService(MessageSender sender) { this.sender = sender; }

// Setter injection
public void setSender(MessageSender sender) { this.sender = sender;
 }

// Field injection (the framework sets the field)
@Inject
private MessageSender sender;
```

Why constructor injection?

Dependencies are visible in the signature

Fields can be `final`

Object is never half-built

Easy to test with plain `new`

DI versus Service Locator

Service Locator: object asks a central lookup

DI: object is handed what it needs

With DI, dependencies are visible

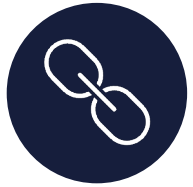
With a locator, they are hidden in the code

What does a DI container do?

Knows which classes exist

Finds out what each class needs

Creates objects in the right order



Connects them together

The general contractor

- Container = general contractor
- Knows which subcontractor does what
- You do not call everyone yourself

Well-known containers

- Spring
- CDI: Weld, Quarkus ArC
- Guice
- At Topicus: CDI

A container is not magic

- It reads what your classes need
- Then it calls new for you
- We build this ourselves this afternoon

The big question

- A container is compiled before it sees your classes
- How can it create a class it does not know yet?

Answer: reflection

Follow-along demo

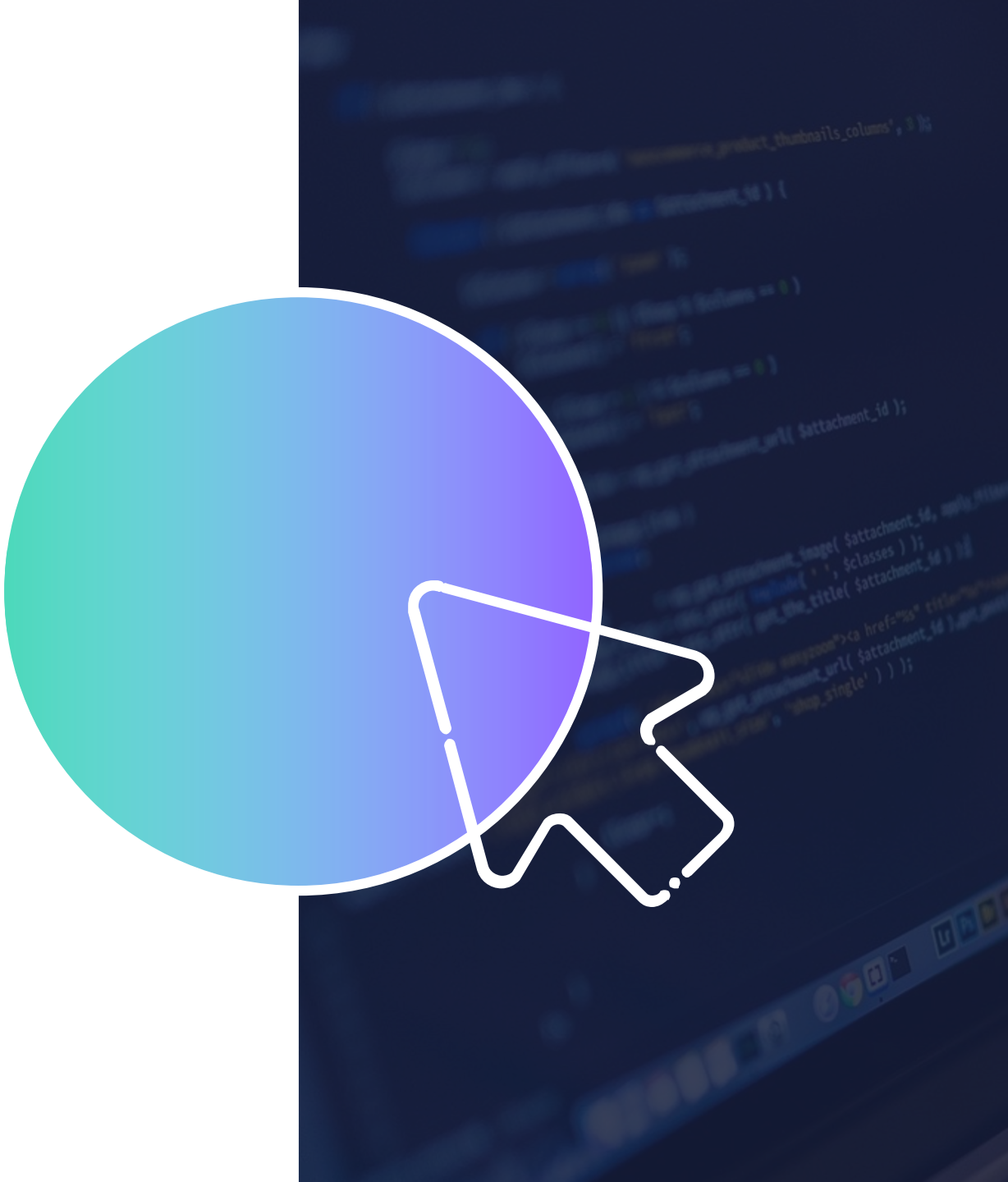
From `new` to Constructor Injection

- Start: `OrderService` creates its own `EmailSender`
- Show: no way to test without sending mail
- Introduce a `MessageSender` interface
- Inject it through the constructor
- Test with a fake `MessageSender`
- Wire by hand in main

Mini exercise 1

Manual wiring

- Refactor to constructor injection
- Wire everything by hand in `main`
- Test `ReportService` with a fake repository
- Note the creation order
- Bonus: what if `A` needs `B` and `B` needs `A`?



Section 2

Annotations

What is an annotation?

- Metadata about code
- A label on a class, method, field or parameter

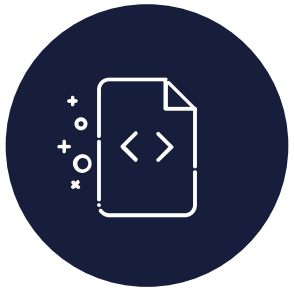
Examples:

- `@Override`
- `@Test`
- `@Path`
- `@GET`

A post-it on a package

- Annotation = a post-it on a package
- The package does not deliver itself
- Something has to read the label

Who reads annotations?



The compiler: `@Override` gives a compile error

Build tools: annotation processors

Runtime: code that uses reflection

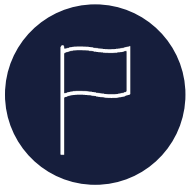
Creating your own annotation

```
public @interface Inject { }

public @interface MaxLength {
    int value();
    String message() default "too long";
}

@MaxLength(50)
private String name;
```

Meta-annotations

- `@Retention(RetentionPolicy.RUNTIME)`
  - Visible at runtime
  - Default is `CLASS`: not visible
- `@Target(ElementType.FIELD)`
  - Where it may be placed
- Also: `CONSTRUCTOR`, `TYPE`, `METHOD`

Marker annotations vs. annotations with values



Marker: `@Inject`, `@ApplicationScoped`

With values: `@MaxLength(50)`, `@Min(18)`

Marker says: this is special

Value says: this is how

Declaration and behavior are separate

Annotation: declares intent

Other code: provides the behavior

Pro:

- Declarative code

Cons:

- Behavior is less visible
- Mistakes show up at runtime

```java
@ApplicationScoped
public class OrderService {

    @Inject
    public OrderService(PaymentService payment) { ... }
}
```

Annotations in our container

`@Inject:` use this constructor

`@ApplicationScoped:` this class is a bean

Section 3

Reflection

What is reflection?

- A program that inspects itself at runtime
- Look at classes, constructors, fields, methods
- Use them without knowing them at compile time

The phone book

Normal code

Call a known number

Reflection

Look it up in the phone book, then call

Why does reflection exist?

- Frameworks do not know your classes
- Spring, Jackson, JUnit, JPA, CDI
- They discover your code at runtime

```
Class<?> a = Foo.class;
Class<?> b = foo.getClass();
void registerClass(Class<?> type) { ... }
```

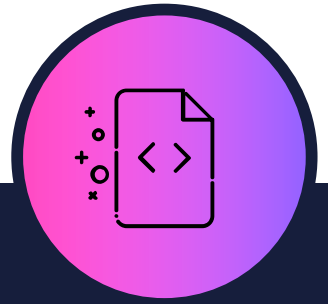
Starting point: Class

A `Class` object holds: name, superclass, interfaces, modifiers, annotations

What can you inspect?

- Constructors: `getDeclaredConstructors()`
- Fields: `getDeclaredFields()`
- Methods: `getDeclaredMethods()`
- Annotations: `isAnnotationPresent(...)`

What can you do with it?



```
Object o = constructor.newInstance(args);
Object v = field.get(obj);
field.set(obj, value);
Object r = method.invoke(obj, args);
```

Private members and setAccessible

- `setAccessible(true)` opens `private` members
- It bypasses encapsulation
- Frameworks do this on purpose

Exceptions in reflection

Many checked exceptions

- `NoSuchMethodException`
- `IllegalAccessException`
- `InstantiationException`

- `InvocationTargetException` wraps the real error
- Read `getCause()`

Trade-offs

Strength: generic code for any class

No compile-time checks

Harder to read

Refactoring tools miss it

Slower than a direct call

Back to the container

- Registry: `Class` to object
- Find the `@Inject` constructor
- Read `getParameterTypes()`
- Find or create those objects
- Call `newInstance()`

Follow-along demo

Dissecting a Class

- A `Person` class with private fields
- Print fields, constructors, methods
- Create an object with `newInstance`
- Read a private field
- Call a method with `invoke`
- Trigger the exceptions

Mini exercise 2

Annotations and reflection, your own validator

- Create `@NotBlank` and `@Min(int value)`
- Annotate private `User` fields `name` and `age`
- Write `Validator.validate(Object obj)`
- Collect errors in a `List<String>`
- Experiment: remove `@Retention(RUNTIME)`
- **Bonus:** `@MaxLength`, superclass fields

Section 4

Generics and type erasure

Why this topic now?

- A container links objects to their type
- Generics make that harder than expected

```java
List<String> names = new ArrayList<>();
names.add("Ada");
String first = names.get(0); // no cast
```

Generics in one minute

- Compile-time type checking
- Generics are a compile-time feature

```
new ArrayList<String>().getClass()
    == new ArrayList<Integer>().getClass() // true
```

Type erasure

- Compiler removes the type parameters
- `List<String>` and `List<Integer>`: both `List` at runtime

Why erasure?

Generics arrived in Java 5

Old code had to keep working

Backwards compatibility with Java 1.4

What you cannot do

- No `new T()`
- No `T.class`
- No `instanceof List<String>` on an unknown type
- No generic array creation

Not all type information is gone

Declarations keep their generic type

Fields, parameters, return types

Superclass and interfaces

Stored in the class file

```
Field f = Foo.class.getDeclaredField("names");
f.getType();        // interface java.util.List
f.getGenericType(); // java.util.List<java.lang.String>
```

getType versus getGenericType

- `ParameterizedType`
- `getActualTypeArguments()`
- Constructors: `getGenericParameterTypes()`

What this means for a container

- Registry: `Map<Class<?>, Object>`
- `List<String>.class` does not exist
- Container sees only `List`
- Frameworks use type tokens, such as `TypeLiteral`
- Today: only non-generic beans
- Day 3: `Provider<T>`

Mini exercise 3

Type erasure in practice

- Compare `getClass()` of `List<String>` and `List<Integer>`
- Print `getType()` and `getGenericType()` of fields
- Write `printTypeArguments(Field f)`
- Think: `List<PaymentService>` or `List<EmailSender>`?
- **Bonus:** generic type of a local variable?

Recap of the morning

IoC and DI: receive, do not create

Constructor injection is the preference

Container: finds, creates and connects

Annotations: metadata, `@Retention`, `@Target`

Reflection: inspect and call at runtime

Type erasure: generics vanish, declarations remain

Looking ahead to the afternoon

- Build your own mini-CDI container
- Registry with manually registered beans
- `@Inject` and `@ApplicationScoped`
- Constructor injection through reflection
- Clear errors for missing and circular dependencies

Your project for this module

Build your own mini-CDI container

One project, all week

Fundamentals first

Today's foundation

- What to create: you register it by hand, for now
- What a class needs: `@Inject` plus reflection
- What to do when it goes wrong: clear errors

Technical stack

- Java 21 or newer, Maven and JUnit
- Standard library only
- No external DI library
- Starter project provided

Goal 1: registry

- `Container with Map<Class<?>, Object>`
- `register(Class<T> type, T instance)`
- `getBean(Class<T> type)` returns `T`
- Unknown type: `ContainerException`
- Tests: register, retrieve, unknown class

Goal 2: annotations and constructor injection

- Create `@Inject` and `@ApplicationScoped`
- `registerClass`: a class, not an instance
- Only registered classes are created
- `@Inject` constructor, else the no-arg one
- Resolve parameters, call `newInstance`
- One instance per bean

Goal 3: error handling

- Missing: `UnsatisfiedDependencyException`
  - Which class needs which type
- Circular: `CircularDependencyException`
  - Show the full path, `A → B → A`
- Both extend `ContainerException`
- Wrap reflection exceptions with context
- Tests for both scenarios

Bonus: interfaces

- `bind(PaymentGateway.class, MockGateway.class)`
- Inject by interface type

Starter project walkthrough

Empty `Container` and annotation package

`example.order`: the working chain

`example.broken`: missing dependency, failing constructor

`example.cycle`: `CycleA` and `CycleB`

Your own test classes: `testbeans`

Running the tests